90-DAY JOURNEY

DAY 07/90

AI SECURITY & RED TEAMING

Day 7 — Web Vulnerabilities in AI Applications

Level: Beginner → Intermediate | **Study time: 3 hours** | **3 parts + hands-on lab**

Phase 1 · Foundation · For students, working professionals & freelancers

IN THIS GUIDE

PART 1 The Web Vulnerability Landscape

PART 2 Core Vulnerability Classes — Root Cause & AI Version

PART 3 Output Handling, Defenses & Testing Mindset

PART 4 Hands-on Practical Worksheet

OBJECTIVE

Revise the core web vulnerability classes — access control, injection, XSS, SSRF, misconfiguration — understand their single root cause, and see exactly how each one reappears when an LLM's output flows into browsers, databases, shells and HTTP clients.

Educational use only — test only with authorization

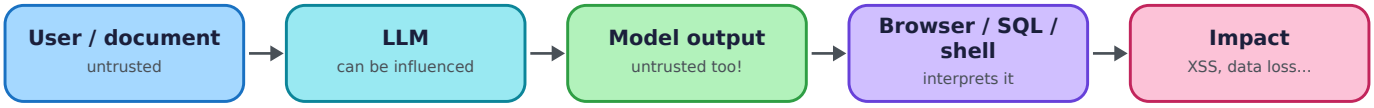
PART 1

The Web Vulnerability Landscape

Hour	Focus	What you do today
HR 1	Learn	OWASP Top 10 + core vulnerability classes and their root causes (Part 1–2).
HR 2	Build	Solve Apprentice-level PortSwigger labs in 3 topics (Part 4, Task 3).
HR 3	Red Team	Run the two local demos: unsafe vs safe output rendering and unsafe vs safe SQL — fix and retest (Tasks 1–2). Last 10 min: write notes in `DAY-XX.md`.

1. Why web vulnerabilities still matter in AI apps

An AI application is still a web application. It has a frontend, APIs, a database and integrations. On top of that, the LLM becomes a **new source of untrusted data**: its output may contain text that a browser, a database or a shell interprets as instructions. Old vulnerabilities come back through a new door.



Treat model output exactly like user input.

> **! KEY POINT**
> OWASP LLM Top 10 lists this as **LLM05: Improper Output Handling**. The fix is mostly classic web security: encoding, parameterization, allow-lists and least privilege.

2. OWASP Top 10 (web) — quick map

Category	In plain words	AI app angle
Broken Access Control	Users act outside their permissions.	Reading other users' chats / documents (Day 6).
Security Misconfiguration	Unsafe defaults, debug mode, open storage.	Exposed notebooks, model servers, vector DBs (Day 2).
Software Supply Chain Failures	Risky or compromised dependencies.	Malicious packages or model files (Day 56–57).
Cryptographic Failures	Sensitive data not protected properly.	API keys / chat logs stored in plain text.
Injection	Untrusted data interpreted as code / commands.	LLM output used in SQL, shell or templates.
Insecure Design	Missing security thinking in the design.	LLM decides authorization; agent with too many tools.
Authentication Failures	Weak login / session handling.	Long-lived API keys, no MFA on admin consoles.
Software / Data Integrity Failures	Trusting unverified code or data.	Unverified model weights, poisoned training / RAG data.
Logging & Alerting Failures	Attacks not detected.	No logging of prompts, tool calls or abuse (Day 37, 64).
Mishandling Exceptional Conditions	Errors fail open or leak details.	Guardrail errors that let output through unchecked.

> **i NOTE**
> This follows the OWASP Top 10:2025 edition. Many courses and reports still use the 2021 list (where SSRF had its own category). The ideas are the same — check owasp.org/Top10 for the current version.

WHO ARE YOU? — HOW TODAY HELPS YOU

● STUDENT

PortSwigger Web Security Academy is free and world-class. Aim for all Apprentice labs in 5 topics by the end of Phase 1 — great for your resume.

● WORKING PRO

Developers at your company build AI features fast. Knowing these classes lets you spot risky designs in reviews before they ship.

● FREELANCER

Clients expect a normal web/API test **plus** AI-specific tests. Covering both makes your offer more complete and valuable.

PART 2

Core Vulnerability Classes — Root Cause & AI Version

1. Injection (SQL, command, template)

	Details
What	Input is concatenated into a query or command, so part of the data is executed as code.
Root cause	Mixing **code and data** in one string.
AI version	Text-to-SQL features, or agents that build shell commands from model output.
Defense	Parameterized queries, no shell (`subprocess` with argument lists), allow-listed operations, least-privilege DB user.

2. Cross-Site Scripting (XSS)

	Details
What	Untrusted data is rendered as HTML / JavaScript in another user's browser.
Root cause	Output inserted into a page **without context-aware encoding**.
AI version	Chat UIs rendering model output as raw HTML or unsafe Markdown; shared conversations shown to other users.
Defense	Render output as text, sanitize Markdown with a trusted library, Content-Security-Policy, restrict image / link domains.

3. Server-Side Request Forgery (SSRF)

	Details
What	The server is made to send requests to destinations the attacker chooses.
Root cause	Server fetches user-influenced URLs without restriction.
AI version	'Summarize this URL', browsing tools, webhook tools, image-from-URL features in agents.
Defense	Allow-list domains, block internal / metadata IP ranges, run fetchers in an isolated network, no redirects to internal hosts.

4. Path traversal and unsafe file handling

	Details
What	User-controlled file names reach files outside the intended folder.
Root cause	File paths built from untrusted input without normalization and checks.
AI version	File-reading tools for agents, document upload / download in RAG apps.
Defense	Use ids instead of paths, resolve and verify the final path stays inside a base folder, sandbox tool file access.

5. Cross-Site Request Forgery (CSRF)

	Details
What	A malicious site makes the victim's browser send an authenticated request.
Root cause	Browsers attach cookies automatically; the server does not verify intent.
AI version	Cookie-authenticated endpoints that trigger agent actions (send email, delete data).
Defense	`SameSite` cookies, CSRF tokens, re-confirmation for sensitive actions.

! REMEMBER

For each class, learn three things: **root cause**, **where it shows up in AI apps**, and **the standard fix**. Payload lists are less important than understanding why a bug exists.

! SCOPE RULE

Test only systems you own or have written permission to test: your own laptop, your own lab VM, intentionally vulnerable practice labs (PortSwigger, OverTheWire, Gandalf) or a client engagement with a signed scope.

PART 3

Output Handling, Defenses & Testing Mindset

1. The output-handling boundary

Every time model output leaves the LLM and enters another component, ask: **how will this component interpret the text?** That boundary is where security controls belong.

Output goes to...	Interpreted as	Required control
Browser (chat UI)	HTML / JS / Markdown	Escape or sanitize; CSP; restrict external images and links.
Database	SQL	Parameterized queries, read-only DB role for AI features, query allow-lists.
Shell / OS	Commands	Avoid shells; fixed commands with validated arguments; sandbox.
HTTP client / tools	URLs, requests	Domain allow-lists, block internal ranges, user confirmation for actions.
Other code (`eval`)	Program code	Never execute model output directly; use structured, validated formats (JSON schema).
Emails / files / tickets	Content for humans + systems	Label as AI-generated, validate, require approval for external sending.

◆ **REAL-WORLD PATTERN**

Several real AI chat products were found rendering Markdown images from model output. Because a Markdown image makes the browser contact an external URL, this could leak conversation data. Vendors fixed it by restricting which image domains may load — a classic output-handling defense.

2. Defense in depth

01 Validate input (types, length, format)	**02** Encode / sanitize output for its context
03 Parameterize every query	**04** Least privilege for DB users, services and agents
05 Allow-lists beat block-lists	**06** Content-Security-Policy in AI web UIs
07 Human approval for high-impact actions	**08** Log and monitor unusual outputs and tool calls

3. How to practice responsibly

01 Use intentionally vulnerable labs: PortSwigger Web Security Academy (free, legal, built for learning).

02 Start with **Apprentice** labs, read the explanation page **before** the lab.

03 After each lab, write: root cause, impact, fix — in your own words.

04 Rebuild the bug and the fix locally in a tiny app (today's Tasks 1–2). Building it teaches more than solving it.

05 Never test real websites without written permission — not even 'just a quick check'.

4. Practice resources

PortSwigger Academy portswigger.net/web-security	**PortSwigger: All labs** portswigger.net/web-security/all-labs	**OWASP Top 10** owasp.org/Top10/
OWASP Cheat Sheets cheatsheetseries.owasp.org	**OWASP LLM05 Output Handling** genai.owasp.org	**MDN: CSP** developer.mozilla.org/docs/Web/HTTP/CSP

> ★ **TAKEAWAY**
>
> Classic web vulnerabilities are alive inside AI apps. Treat model output as untrusted, find every place it is interpreted, and apply the standard fix at that boundary.

PART 4

Hands-on Practical Worksheet

LAB	TIME	ENVIRONMENT
Output handling + SQL demos and PortSwigger Apprentice labs	75–90 minutes	Python + Flask + sqlite3 on localhost; PortSwigger Web Security Academy (free account)

Task 1 — Unsafe vs safe output rendering (your own local demo)

A tiny page that displays a 'model reply'. Version A renders it as HTML, version B as text.

render_demo.py

```
# render_demo.py  ->  python render_demo.py   (127.0.0.1:5001)
from flask import Flask, request, render_template_string
app = Flask(__name__)
PAGE = "<h3>AI reply</h3><div>{{ reply %s }}</div>"

@app.get("/unsafe")
def unsafe():   # renders reply as raw HTML
    return render_template_string(PAGE % "| safe", reply=request.args.get("r", ""))

@app.get("/safe")
def safe():     # Jinja auto-escaping: reply shown as plain text
    return render_template_string(PAGE % "", reply=request.args.get("r", ""))

app.run(host="127.0.0.1", port=5001)
```

Open both pages with the harmless test value `?r=<i>CANARY-07</i>` (your browser will URL-encode it).

Rendering results

/unsafe shows: _____

/safe shows: _____

Why is /unsafe dangerous in a chat app? _____

OWASP LLM id: _____

Fix in one line: _____

Task 2 — Unsafe vs safe SQL (your own local demo)

A student search that builds SQL with string formatting. Search for a normal name with an apostrophe.

● sql_demo.py

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE students(name TEXT, marks INT)")
db.executemany("INSERT INTO students VALUES (?, ?)", [("Priya", 91), ("O'Brien", 78)])

def search_unsafe(name):
    return db.execute(f"SELECT * FROM students WHERE name = '{name}'").fetchall()

def search_safe(name):
    return db.execute("SELECT * FROM students WHERE name = ?", (name,)).fetchall()

for fn in (search_unsafe, search_safe):
    try:
        print(fn.__name__, fn("O'Brien"))
    except Exception as e:
        print(fn.__name__, "ERROR:", e)
```

SQL results

search_unsafe output: _____

search_safe output: _____

What does the error tell you? _____

If an LLM generated `name`, what changes? _____

Task 3 — PortSwigger Apprentice labs

Solve at least one Apprentice lab in three topics (e.g. access control, SQL injection, XSS, path traversal, SSRF).

Topic	Lab name	Root cause (own words)	Fix

Task 4 — Map web bugs to AI features

For each AI feature, write the classic vulnerability class it could introduce and the control you would check.

AI feature	Possible vulnerability class	Control to check
Chat UI rendering Markdown		
Text-to-SQL report builder		
'Summarize this URL' tool		
Agent file-reader tool		
'Send email' agent action		

> ⚠ **HONESTY RULE**
> If you have not run a test, write **Not tested yet** in the evidence and retest fields. Do not invent results. Real red teamers are trusted because their evidence is real.

Task 5 — Self-check quiz

Answer without looking, then check the key at the end.

Q1. What is the single root cause shared by SQL injection, command injection and XSS?

Q2. Why does OWASP say LLM output must be treated as untrusted?

Q3. Your chatbot shows `CANARY-07` in italics after you typed it inside `<i>` tags. What does this prove?

Q4. What is the correct fix for SQL built from user or model input?

Q5. An AI feature fetches any URL a user gives it. Which vulnerability class should you think about?

Task 6 — End-of-day checklist

- [ ] Finished all three hours (learn, build, red team).
- [ ] Ran the lab myself — not just read about it.
- [ ] Ran both local demos and saw the unsafe vs safe difference myself.
- [ ] Solved Apprentice labs in at least three PortSwigger topics.
- [ ] Completed the AI feature → vulnerability map.
- [ ] Separated assumptions from confirmed findings.
- [ ] Recorded evidence honestly (no invented results).
- [ ] Wrote my `DAY-XX.md` notes and pushed them to my notes repo / folder.
- [ ] Ticked today in the 90-Day tracker.

● **DAY-07.md (copy into your notes repo)**

```
# DAY-07.md — Web Vulnerabilities in AI Applications
Date:            ____ / ____ / ________
Time spent:      Learn __ min | Build __ min | Red team __ min

## 3 things I learned
1.
2.
3.

## What I built / ran (commands, code, screenshots)
-

## Findings (hypothesis -> test -> evidence -> fix -> retest)
-

## Doubts to research tomorrow
-
```

✓ **ANSWER KEY — CHECK ONLY AFTER YOU TRY**

Q1. Untrusted data is mixed into something that gets interpreted as code (SQL, shell, HTML/JS) without proper separation or encoding.

Q2. The output can be influenced by user input or poisoned documents. If it is passed into a browser, database, shell or HTTP client unsafely, classic vulnerabilities appear (OWASP LLM05).

Q3. The UI renders model/user output as HTML instead of plain text — an improper output handling weakness that can lead to XSS.

Q4. Parameterized queries (placeholders), plus a least-privilege database account. Never build SQL with string formatting.

Q5. SSRF — the server could be made to reach internal services. Defense: allow-lists, block internal ranges, separate network for fetchers.